	Pimpri Chinchwad Education Trust's **Pimpri Chinchwad College of Engineering** Department of Information Technology	
ASSIGNMENT 8		

NAME	Viraj Shrikant Pathare
PRN	124B1F149
CLASS	SY IT
DIVISION	A

1. Employee Appraisal System

Collection: employees

```
{
  emp_id: 101,
  name: "Amit",
  age: 30,
  dept: "Engineering",
  performance_score: 85
}
```

---

2. Aggregation in MongoDB

Aggregation is used to process data records and return computed results using stages like $match, $group, $sort, $project, etc.

Example 1: Find Average Performance Score Department-wise

```
db.employees.aggregate([
  {
    $group: {
      _id: "$dept",
      avgScore: { $avg: "$performance_score" }
    }
  }
])
```

Output:

```
{ "_id": "Engineering", "avgScore": 82 }
{ "_id": "HR", "avgScore": 78 }
```

Example 2: Filter + Group (High Performers per Department)

```
db.employees.aggregate([
  {
    $match: {
      performance_score: { $gt: 80 }
    }
  },
  {
    $group: {
      _id: "$dept",
      totalHighPerformers: { $sum: 1 }
    }
  }
])
```

Example 3: Sorting Results (Best Departments First)

```
db.employees.aggregate([
  {
    $group: {
      _id: "$dept",
      avgScore: { $avg: "$performance_score" }
    }
  },
  {
    $sort: { avgScore: -1 }
  }
])
```

Example 4: Projection (Assign Appraisal Grade)

```
db.employees.aggregate([
  {
    $project: {
      name: 1,
      dept: 1,
      grade: {
        $cond: { if: { $gte: ["$performance_score", 80] }, then: "A", else: "B" }
      }
    }
  }
])
```

Example 5: Aggregation with Join ($lookup)

Collection: departments

```
{ dept: "Engineering", manager: "Dr. Sharma" }
db.employees.aggregate([
  {
    $lookup: {
      from: "departments",
      localField: "dept",
      foreignField: "dept",
      as: "dept_info"
```

```
    }
  }
])
```

---

3. Indexing in MongoDB

Indexing improves query performance by allowing faster data retrieval across the employees and appraisals collections.

Create Index on Single Field

```
db.employees.createIndex({ emp_id: 1 })
```

1 = Ascending order

Create Index on Multiple Fields (Compound Index)

```
db.employees.createIndex({ dept: 1, performance_score: -1 })
```

Create Unique Index

```
db.employees.createIndex({ emp_id: 1 }, { unique: true })
```

Prevents duplicate employee IDs

View Indexes

```
db.employees.getIndexes()
```

Drop Index

```
db.employees.dropIndex({ emp_id: 1 })
```

---

4. Performance Example (With vs Without Index)

Without Index:

```
db.employees.find({ emp_id: 101 })
```

Scans entire collection (slow)

With Index:

```
db.employees.createIndex({ emp_id: 1 })
```

Direct lookup (fast)

---

5. Combined Use (Aggregation + Index)

```
db.employees.aggregate([
  {
    $match: { dept: "Engineering" }
  },
  {
    $group: {
      _id: "$dept",
      avgScore: { $avg: "$performance_score" }
```

```
    }
  }
])
```

Index on dept improves $match stage performance:

```
db.employees.createIndex({ dept: 1 })
```

Summary

Concept	Description
Aggregation	Data processing using pipeline stages
$match	Filter employees by condition
$group	Group and calculate appraisal values
$sort	Sort results by rating or score
$project	Select/transform fields
$lookup	Join employees with appraisals/departments
Indexing	Improves query performance